Имя	Акулов Роман
Группа	ЭФМО-01-26

Практическая №5: работа с PostgreSQL через `database/sql`

Тема: Подключение к PostgreSQL через database/sql. Выполнение простых запросов (INSERT, SELECT)

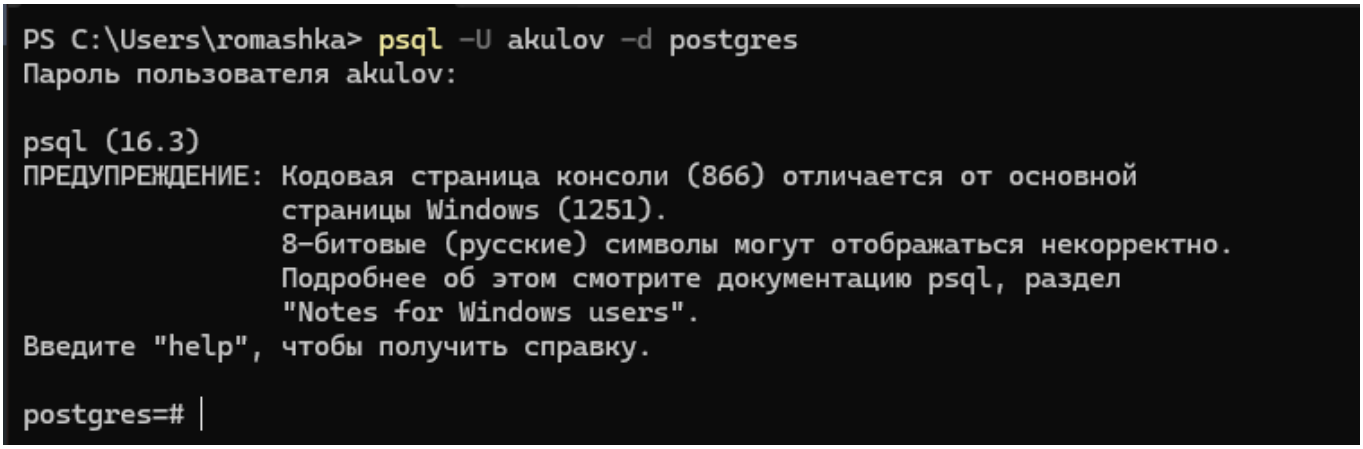
Цели:

1. Установить и настроить PostgreSQL локально.
2. Подключиться к БД из Go с помощью `database/sql` и драйвера PostgreSQL.
3. Выполнить параметризованные запросы INSERT и SELECT.
4. Научиться работать с context, пулом соединений и обработкой ошибок.

1. Ход работы

1.1. Установка и проверка PostgreSQL

Подключимся к серверу PostgreSQL через PowerShell и убедимся, что можем выполнять команды в консоли.



1.2. Создание БД и таблицы

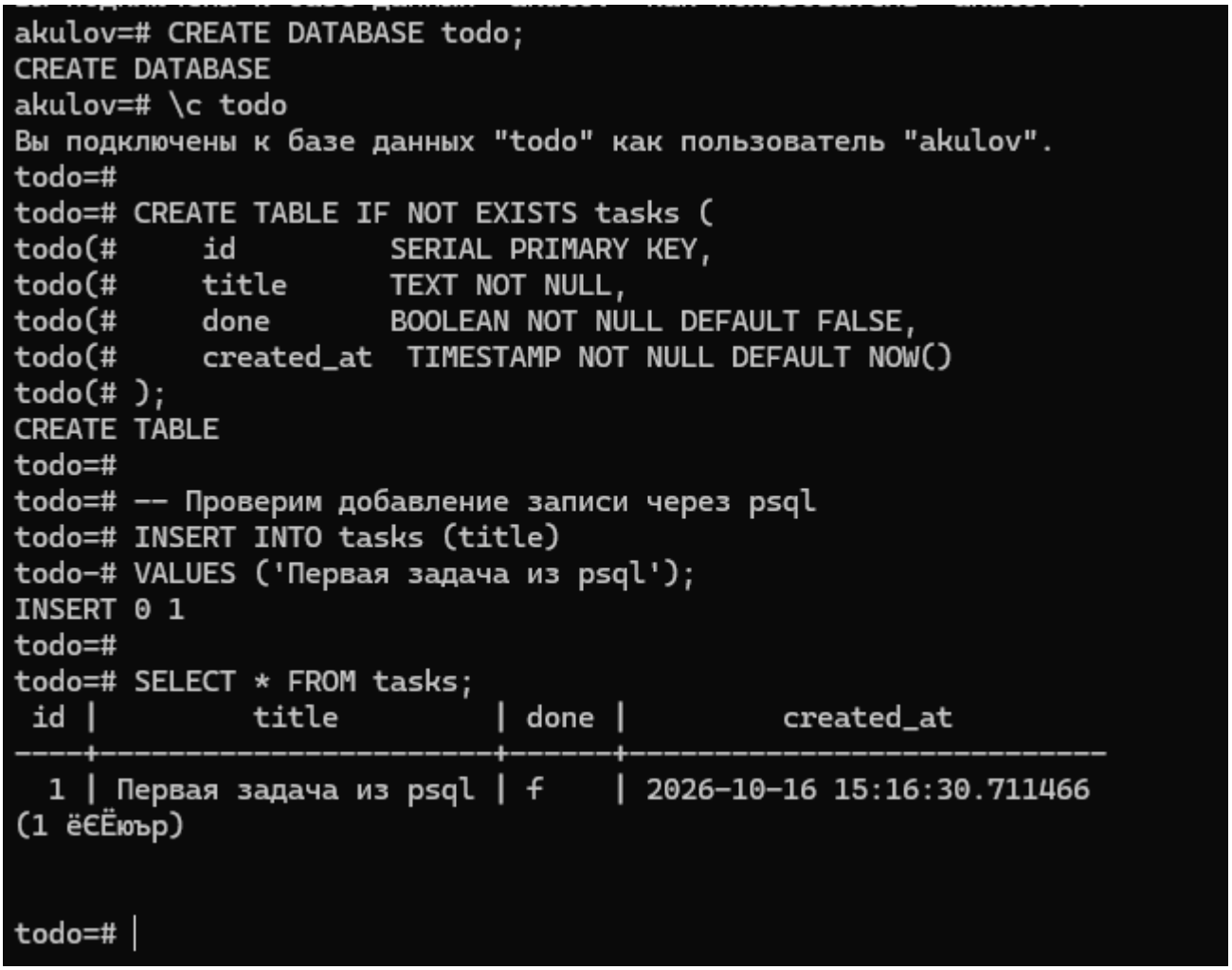
Создадим базу данных `todo` и таблицу `tasks`. В таблице будем хранить идентификатор задачи, её название, статус выполнения и время создания. Для проверки добавим одну запись вручную через `psql`.

```sql
CREATE DATABASE todo;
\c todo

CREATE TABLE IF NOT EXISTS tasks (
    id          SERIAL PRIMARY KEY,
    title       TEXT NOT NULL,
    done        BOOLEAN NOT NULL DEFAULT FALSE,
    created_at  TIMESTAMP NOT NULL DEFAULT NOW()
);

-- Проверим добавление записи через psql
INSERT INTO tasks (title)
VALUES ('Первая задача из psql');

SELECT * FROM tasks;
```



После выполнения запроса проверим, что таблица создана, а добавленная задача отображается в результатах SELECT.

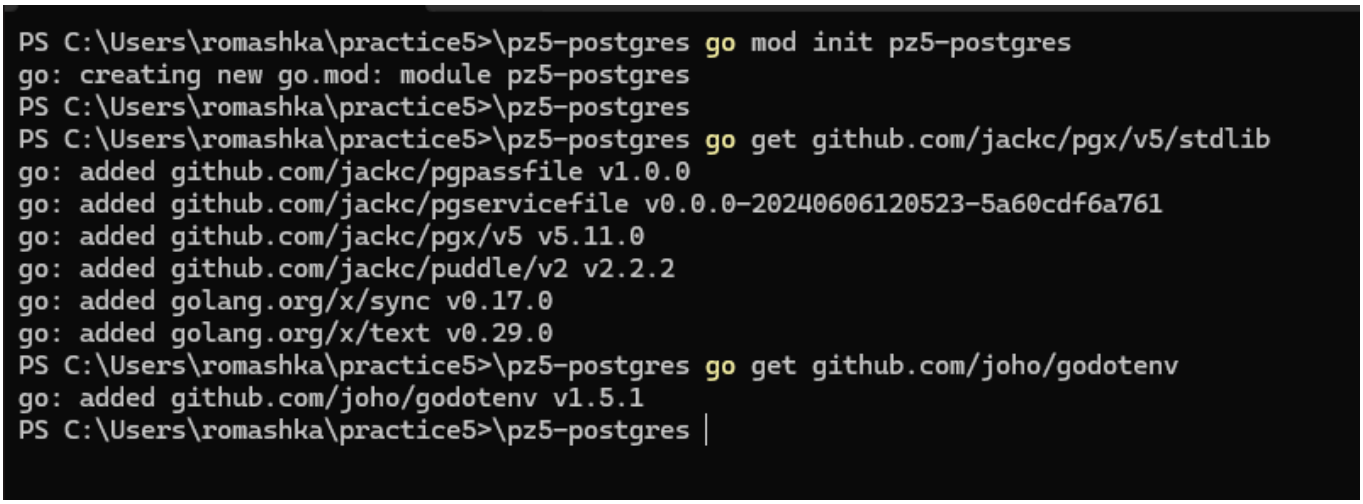
1.3. Создание Go-проекта

Создадим отдельную папку для проекта и инициализируем Go-модуль. Затем подключим драйвер PostgreSQL для `database/sql` и библиотеку `godotenv`, которая позволит загружать настройки из файла `.env`.

```
mkdir pz5-postgres
cd pz5-postgres

go mod init pz5-postgres

go get github.com/jackc/pgx/v5/stdlib
go get github.com/joho/godotenv
```



В проекте разместим код подключения к БД, методы работы с задачами и точку входа в приложение.

```
pz5-postgres/
├── go.mod
├── go.sum
├── .env
├── db.go
├── repository.go
└── main.go
```

В `.env` укажем строку подключения к созданной базе данных. Здесь используются локальный сервер PostgreSQL, пользователь `akulov` и база `todo`.

```
DATABASE_URL=postgres://akulov:admin@localhost:5432/todo?sslmode=disable
```

> ⚠ **Настройки подключения Файл `.env` содержит пароль от базы данных. Не будем добавлять его в публичный Git-репозиторий. Для этого можно указать `.env` в `.gitignore`, а в репозитории оставить пример конфигурации без настоящих учётных данных.**

1.4. Код проекта

1.4.1. `db.go`

В этом файле реализуем функцию `openDB`, которая открывает подключение к PostgreSQL через `database/sql`. Настроим пул соединений и проверим доступность сервера с помощью `PingContext`.

```
package main

import (
    "context"
    "database/sql"
    "log"
    "time"

    // Регистрируем драйвер pgx для database/sql
    _ "github.com/jackc/pgx/v5/stdlib"
)

func openDB(dsn string) (*sql.DB, error) {
    db, err := sql.Open("pgx", dsn)
    if err != nil {
        return nil, err
    }

    // Настраиваем пул соединений
    db.SetMaxOpenConns(10)
    db.SetMaxIdleConns(5)
    db.SetConnMaxLifetime(30 * time.Minute)

    // Проверяем подключение с ограничением по времени
    ctx, cancel := context.WithTimeout(
        context.Background(),
        2*time.Second,
    )
    defer cancel()

    if err := db.PingContext(ctx); err != nil {
```

```
        return nil, err
    }

    log.Println("Connected to PostgreSQL")
    return db, nil
}
```

1.4.2. `repository.go`

Создадим модель `Task` для представления задач в Go и структуру `Repo`, через которую будем выполнять запросы к БД.

Метод `CreateTask` добавляет задачу и возвращает её идентификатор. Для передачи названия используем параметр `$1`, чтобы не собирать SQL-запрос из пользовательской строки.

```
package main

import (
    "context"
    "database/sql"
    "time"
)

// Task — модель для сканирования результатов SELECT
type Task struct {
    ID        int
    Title     string
    Done      bool
    CreatedAt time.Time
}

type Repo struct {
    DB *sql.DB
}

func NewRepo(db *sql.DB) *Repo {
    return &Repo{DB: db}
}

// CreateTask — параметризованный INSERT с возвратом id
func (r *Repo) CreateTask(
    ctx context.Context,
    title string,
) (int, error) {
```

```go
    var id int

    const q = `
        INSERT INTO tasks (title)
        VALUES ($1)
        RETURNING id;
    `

    err := r.DB.QueryRowContext(ctx, q, title).Scan(&id)
    return id, err
}

// ListTasks — базовый SELECT всех задач
func (r *Repo) ListTasks(ctx context.Context) ([]Task, error) {
    const q = `
        SELECT id, title, done, created_at
        FROM tasks
        ORDER BY id;
    `

    rows, err := r.DB.QueryContext(ctx, q)
    if err != nil {
        return nil, err
    }
    defer rows.Close()

    var out []Task

    for rows.Next() {
        var t Task

        if err := rows.Scan(
            &t.ID,
            &t.Title,
            &t.Done,
            &t.CreatedAt,
        ); err != nil {
            return nil, err
        }

        out = append(out, t)
    }

    return out, rows.Err()
}
```

```go
// ---------- Задание 1: ListDone ----------

func (r *Repo) ListDone(
    ctx context.Context,
    done bool,
) ([]Task, error) {
    const q = `
        SELECT id, title, done, created_at
        FROM tasks
        WHERE done = $1
        ORDER BY id;
    `

    rows, err := r.DB.QueryContext(ctx, q, done)
    if err != nil {
        return nil, err
    }
    defer rows.Close()

    var out []Task

    for rows.Next() {
        var t Task

        if err := rows.Scan(
            &t.ID,
            &t.Title,
            &t.Done,
            &t.CreatedAt,
        ); err != nil {
            return nil, err
        }

        out = append(out, t)
    }

    return out, rows.Err()
}

// ---------- Задание 2: FindByID ----------

func (r *Repo) FindByID(
    ctx context.Context,
    id int,
) (*Task, error) {
    const q = `
```

```
        SELECT id, title, done, created_at
        FROM tasks
        WHERE id = $1;
    `

    var t Task

    err := r.DB.QueryRowContext(ctx, q, id).Scan(
        &t.ID,
        &t.Title,
        &t.Done,
        &t.CreatedAt,
    )
    if err != nil {
        return nil, err
    }

    return &t, nil
}

// ---------- Задание 3: CreateMany (через транзакцию) ----------

func (r *Repo) CreateMany(
    ctx context.Context,
    titles []string,
) error {
    tx, err := r.DB.BeginTx(ctx, nil)
    if err != nil {
        return err
    }

    // Откатываем транзакцию, если до Commit возникла ошибка
    defer tx.Rollback()

    const q = `INSERT INTO tasks (title) VALUES ($1);`

    stmt, err := tx.PrepareContext(ctx, q)
    if err != nil {
        return err
    }
    defer stmt.Close()

    for _, title := range titles {
        if _, err := stmt.ExecContext(ctx, title); err != nil {
            return err
        }
```

```
    }

    return tx.Commit()
}
```

1.4.3. `main.go`

В `main.go` загрузим параметры подключения из `.env`, откроем базу и создадим репозиторий. Для выполнения операций зададим общий контекст с таймаутом в пять секунд.

Сначала добавим несколько задач через `CreateTask`, затем выведем все записи с помощью `ListTasks`. После этого проверим дополнительные методы: `ListDone`, `FindByID` и `CreateMany`.

```
package main

import (
    "context"
    "fmt"
    "log"
    "os"
    "time"

    "github.com/joho/godotenv"
)

func main() {
    // Если .env отсутствует, используем переменные окружения
    _ = godotenv.Load()

    dsn := os.Getenv("DATABASE_URL")
    if dsn == "" {
        // Настройки для локального учебного стенда
        dsn = "postgres://postgres:admin@localhost:5432/todo?sslmode=disable"
    }

    db, err := openDB(dsn)
    if err != nil {
        log.Fatalf("openDB error: %v", err)
    }
    defer db.Close()

    repo := NewRepo(db)
```

```
    ctx, cancel := context.WithTimeout(
        context.Background(),
        5*time.Second,
    )
    defer cancel()

    // 1) INSERT — добавляем задачи по одной
    titles := []string{
        "Сделать ПЗ №5",
        "Купить кофе",
        "Проверить отчёты",
    }

    for _, title := range titles {
        id, err := repo.CreateTask(ctx, title)
        if err != nil {
            log.Fatalf("CreateTask error: %v", err)
        }

        log.Printf("Inserted task id=%d (%s)", id, title)
    }

    // 2) SELECT — получаем все задачи
    tasks, err := repo.ListTasks(ctx)
    if err != nil {
        log.Fatalf("ListTasks error: %v", err)
    }

    fmt.Println("=== Tasks ===")
    for _, t := range tasks {
        fmt.Printf(
            "%d | %-24s | done=%-5v | %s\n",
            t.ID,
            t.Title,
            t.Done,
            t.CreatedAt.Format(time.RFC3339),
        )
    }

    // 3) Задание 1: ListDone
    notDone, err := repo.ListDone(ctx, false)
    if err != nil {
        log.Fatalf("ListDone error: %v", err)
    }

    fmt.Println("\n=== ListDone(false) ===")
```

```
    for _, t := range notDone {
        fmt.Printf("%d | %s | done=%v\n", t.ID, t.Title, t.Done)
    }

    // 4) Задание 2: FindByID
    if len(tasks) > 0 {
        t, err := repo.FindByID(ctx, tasks[0].ID)
        if err != nil {
            log.Fatalf("FindByID error: %v", err)
        }

        fmt.Printf("\n=== FindByID(%d) ===\n", t.ID)
        fmt.Printf(
            "ID=%d Title=%q Done=%v CreatedAt=%s\n",
            t.ID,
            t.Title,
            t.Done,
            t.CreatedAt.Format(time.RFC3339),
        )
    }

    // 5) Задание 3: CreateMany
    batch := []string{
        "Задача A",
        "Задача B",
        "Задача C",
    }

    if err := repo.CreateMany(ctx, batch); err != nil {
        log.Fatalf("CreateMany error: %v", err)
    }

    log.Printf("CreateMany inserted %d tasks", len(batch))
}
```

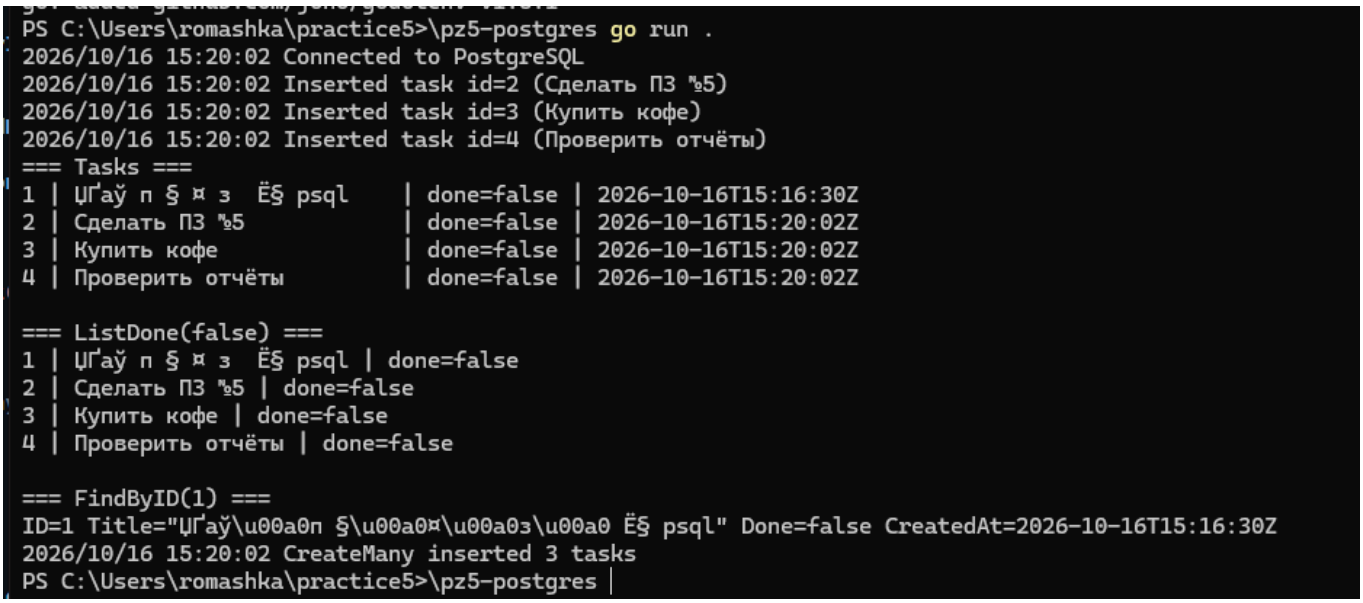


1.5. Запуск приложения

Запустим приложение из корневой папки проекта:

```
go run .
```

При успешном подключении программа добавит задачи в PostgreSQL и выведет их список. В консоли также появятся результаты фильтрации по статусу, поиска по идентификатору и пакетного добавления записей.



Проверим результат непосредственно в PostgreSQL через `psql`. Убедимся, что задачи, добавленные из Go, действительно сохранились в таблице.

```
todo=# SELECT * FROM tasks;
 id |         title          | done |         created_at
----+------------------------+------+----------------------------
  1 | Первая задача из psql  | f    | 2026-10-16 15:16:30.711466
  2 | ╤фхырЄ№ ╧╢ ╢5          | f    | 2026-10-16 15:20:02.909835
  3 | ╧єяшЄ№ ъюЇх             | f    | 2026-10-16 15:20:02.911461
  4 | ╧ЁютхЁшЄ№ юЄўҐЄ√        | f    | 2026-10-16 15:20:02.911996
  5 | ╟рфрўр A                | f    | 2026-10-16 15:20:02.932387
  6 | ╟рфрўр B                | f    | 2026-10-16 15:20:02.932387
  7 | ╟рфрўр C                | f    | 2026-10-16 15:20:02.932387
(7 ёЄЁюъ)
```